Lab 08 Report

CS-301L | Full Stack Web Development

Name: Fatima Akbar Roll No.: 242717

20 October 2026

A complete NestJS application was implemented in the initially empty lab08 repository. It includes Students, Courses and Fees modules, constructor injection, configurable storage, fee policies, DTO validation and automated tests. Screenshots in this report show real command/API output rendered for readability; they are not Postman desktop screenshots.

Task 1 — Courses and dependency injection

CoursesModule registers a Symbol COURSE_STORE through useFactory. APP_CONFIG is a string token, CreditService is a class provider, and the abstract Logger class is aliased to AppLogger with useExisting. Storage can be switched between an isolated seeded memory store and a JSON file store. CoursesService implements list, statistics, lookup, create, update and delete, logging each creation and deletion.

Task 2 — Pluggable fees

FEE_RATE is supplied with useValue (5000). FEE_POLICY uses a factory to choose standard or merit pricing. The FEE_CALCULATOR alias resolves the same FeesService instance. The same request for Sara Ahmed at 12 credits returns 60000 under standard pricing and 30000 under merit pricing; the service source is unchanged between runs.

Task 3 — DTO validation

The global ValidationPipe transforms DTOs and rejects unknown properties. Course code and strings are cleaned, tags are lowercased, and field constraints return custom details. Numeric course JSON strings are rejected, while numeric fee/query strings are explicitly converted. PUT requires all mandatory non-code fields; PATCH accepts partial fields. RESULTS.md contains actual request and response bodies with decorator explanations.

Task 4 — Automated checks and Postman collection

The project includes service, fee-policy and DTO unit tests, plus HTTP end-to-end tests using an overridden memory store. The exported Lab 08 Tasks collection has 21 requests, including negative checks, status/body assertions, a saved newCode variable

and cleanup. It was executed twice consecutively with Newman, the Postman command-line runner. Import it into Postman to capture a desktop Collection Runner screenshot if your instructor requires that specific UI.

Short answers

(a) Why can an interface not be an injection token? TypeScript interfaces are erased during compilation and have no runtime identity. Nest needs a runtime value such as a string, Symbol or class to look up a provider.

(b) What does singleton mean? A default-scope provider has one shared instance in the application context. Consumers resolving the same provider receive that instance rather than constructing a new service for every request.

(c) How can OtherModule use CoursesService? Add CoursesService to the exports array of CoursesModule. Import CoursesModule in OtherModule and inject CoursesService through the consumer's constructor.

Run and verify

npm ci; start with STORAGE=memory LOG_TO_FILE=true FEE_POLICY=standard npm run start:dev. Stop and restart with FEE_POLICY=merit to compare fees. Run npm test, npm run test:e2e, npm run test:postman and npm run build from the project directory. The complete source project accompanies this report.

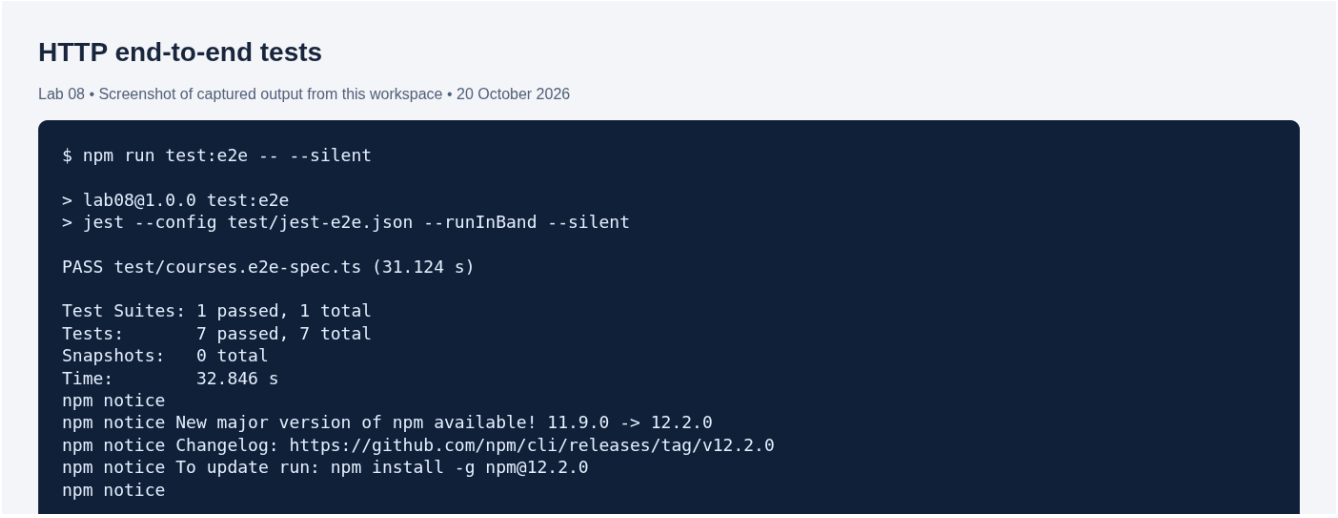
Unit tests

Real output captured from this workspace and rendered for readability. Newman summaries are excerpts, not a Postman desktop UI.



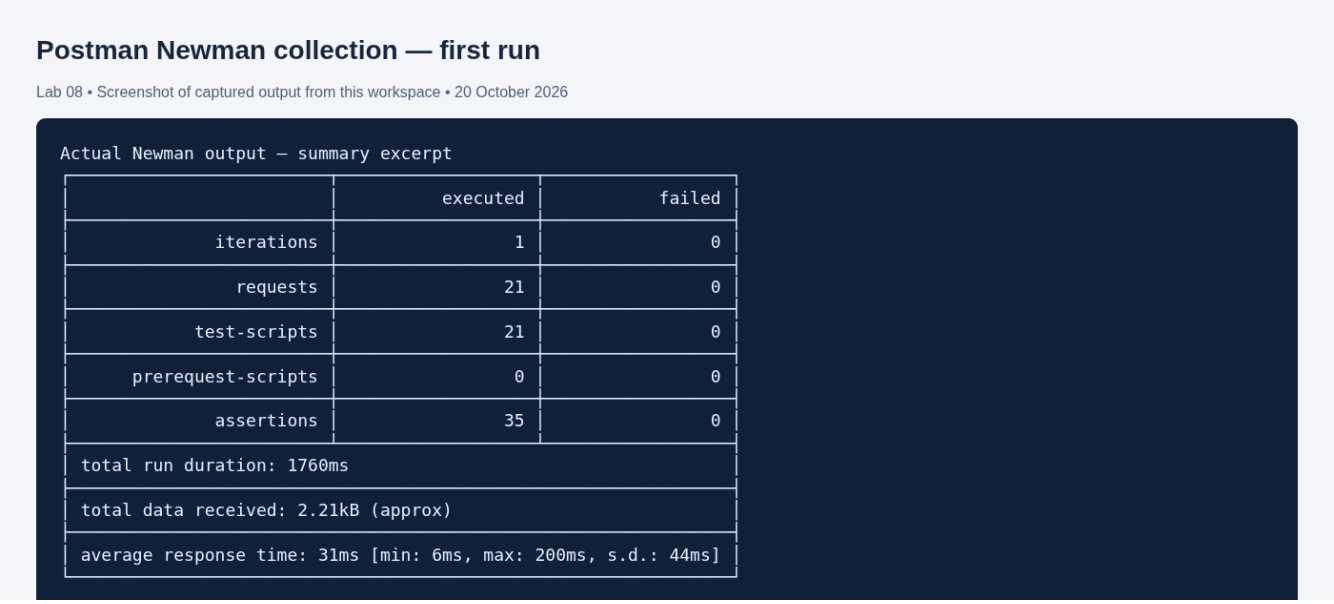
HTTP end-to-end tests

Real output captured from this workspace and rendered for readability. Newman summaries are excerpts, not a Postman desktop UI.



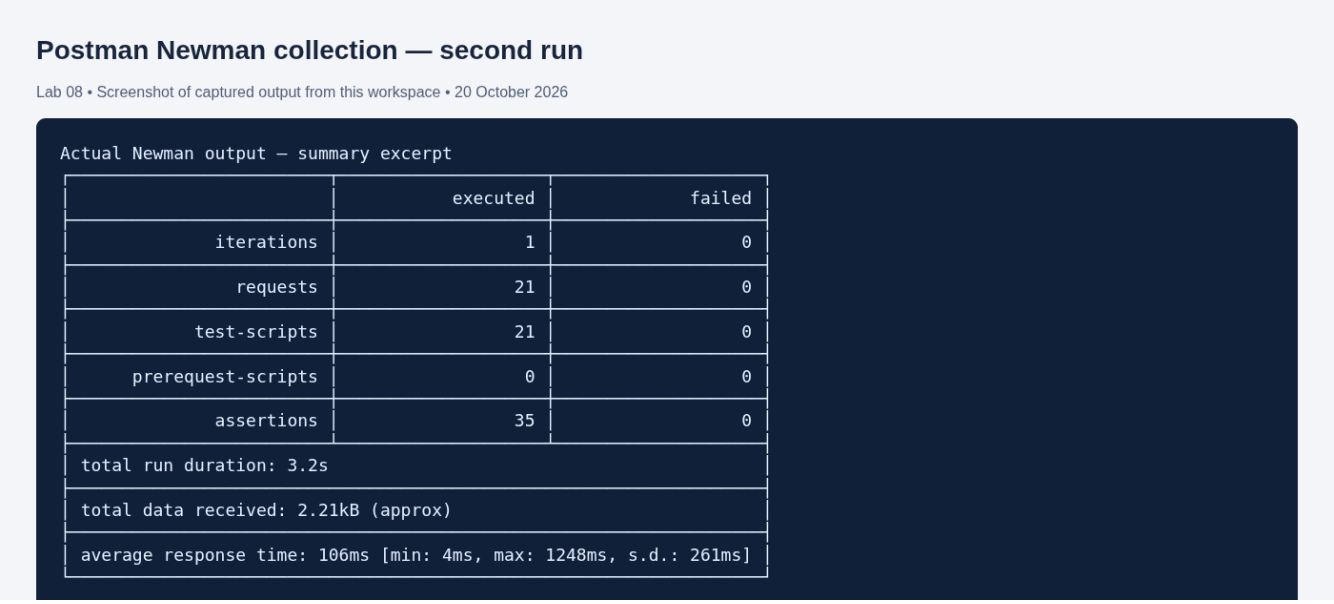
Postman Newman collection — first run

Real output captured from this workspace and rendered for readability. Newman summaries are excerpts, not a Postman desktop UI.



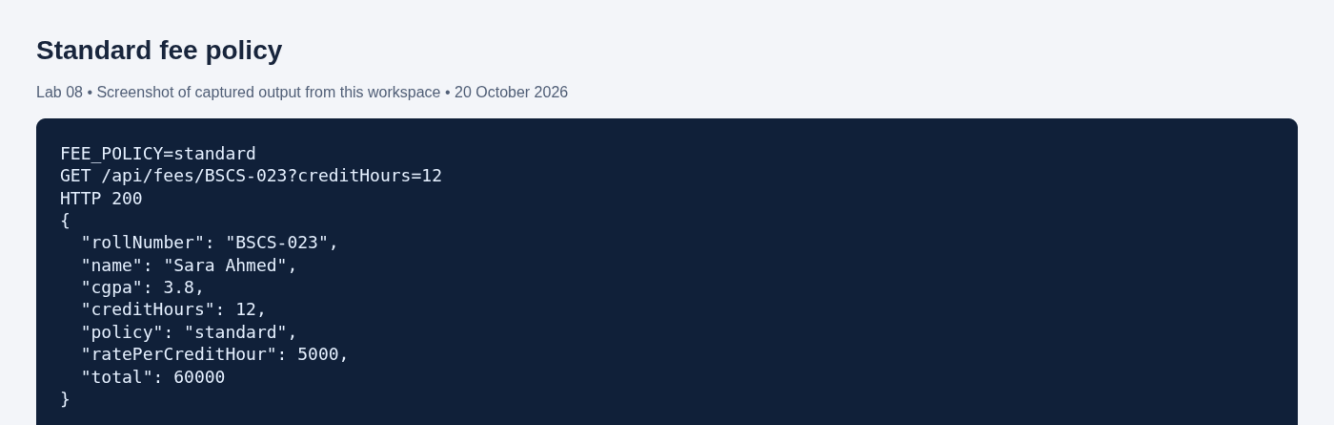
Postman Newman collection — second run

Real output captured from this workspace and rendered for readability. Newman summaries are excerpts, not a Postman desktop UI.



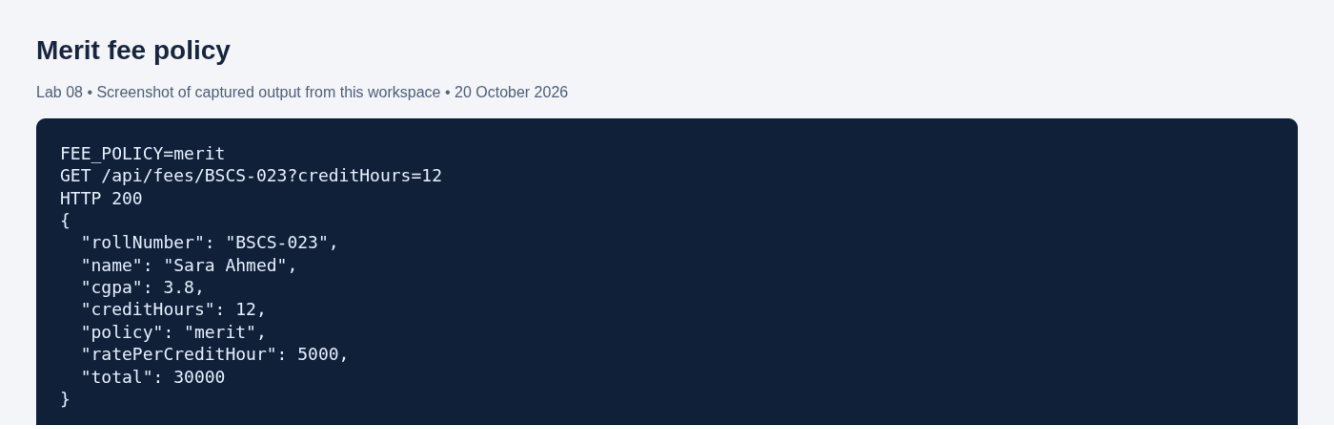
Standard fee policy

Real output captured from this workspace and rendered for readability. Newman summaries are excerpts, not a Postman desktop UI.



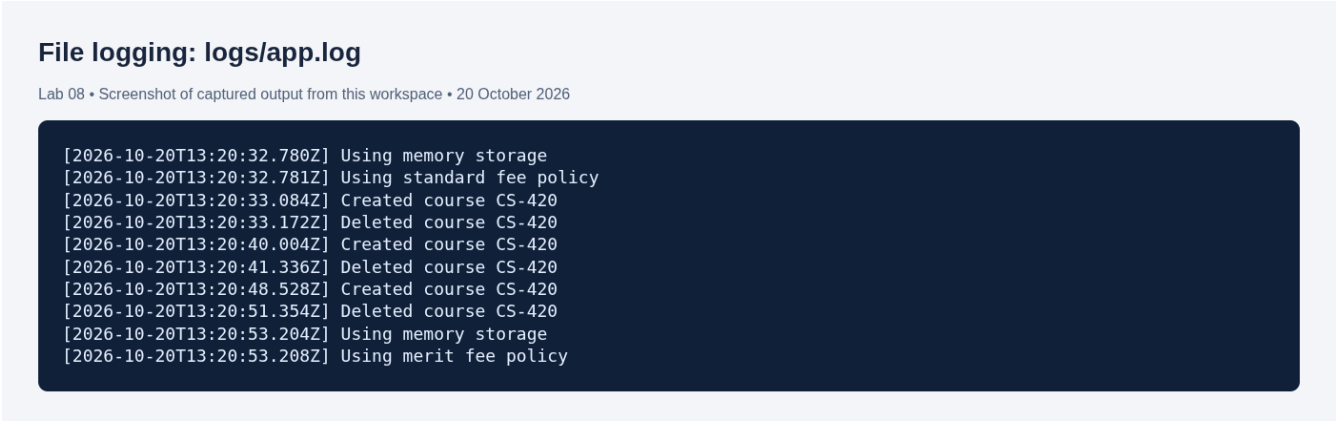
Merit fee policy

Real output captured from this workspace and rendered for readability. Newman summaries are excerpts, not a Postman desktop UI.



File logging: logs/app.log

Real output captured from this workspace and rendered for readability. Newman summaries are excerpts, not a Postman desktop UI.



Validation: invalid course response

Real output captured from this workspace and rendered for readability. Newman summaries are excerpts, not a Postman desktop UI.

